



İSTANBUL TOPKAPI ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

2025-2026 GÜZ DÖNEMİ

VERİ MADENCİLİĞİ (FET445) FİNAL PROJE RAPORU

KONU: SPOTIFY BÜYÜK VERİ SETİ ÜZERİNDE ŞARKI POPÜLARİTESİ TAHMİNİ
VE GELİŞMİŞ MODELLEME

GRUP İSMİ: AKATSUKİ

PROJE GITHUB LİNKİ:

<https://github.com/ahsenhatipoglu/veri-madenciligi-spotify-analiz>

HAZIRLAYANLAR:

AHSEN HATİPOĞLU - 22040301055

ALİ EREN KAMALIOĞLU - 22040301081

1. GİRİŞ VE PROJE AMACI

Müzik endüstrisindeki 'Hit' (Popüler) kavramı; değişen tüketim alışkanlıkları, dijital platform algoritmaları ve şarkıların teknik akustik yapıları gibi birçok faktörden etkilenecek sürekli bir dinamizm göstermektedir. Vize aşamasında temelleri atılan bu proje, Kaggle platformundan elde edilen 1.2 milyon satırlık veri setindeki teknik ses analizlerini kullanarak, bir şarkının potansiyel popülaritesini önceden tahmin edebilen bir veri madenciliği modeli geliştirmeyi amaçlamaktadır.

Final projesi kapsamında; vize döneminde kurulan temel (baseline) modellere ek olarak, **PyTorch kütüphanesi ile Derin Öğrenme (Deep Learning)** mimarileri geliştirilmiş, **XGBoost ve LightGBM** gibi gelişmiş topluluk (ensemble) öğrenme algoritmaları projeye entegre edilmiş ve detaylı hiperparametre optimizasyonları yapılmıştır.

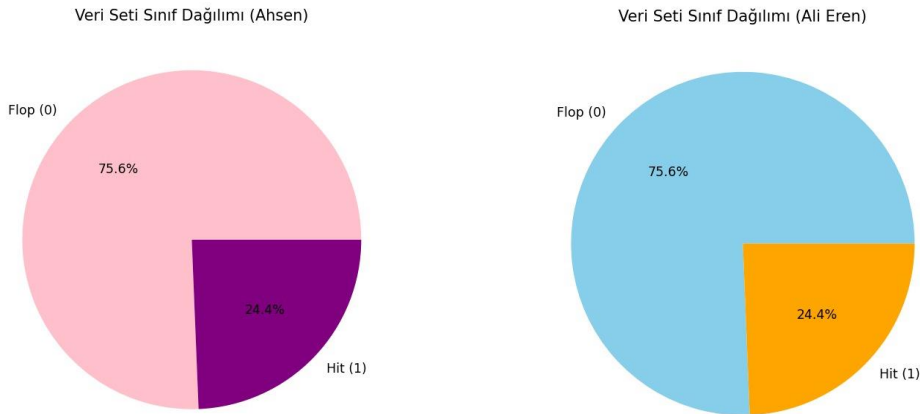
2. VERİ SETİ VE ÖN İŞLEME

Projede kullanılan veri seti, şarkıların "dans edilebilirlik", "enerji", "akustiklik", "süre" gibi sayısal özelliklerini içermektedir.

Veri Kaynağı: Kaggle - Spotify 1.2M+ Songs Dataset

2.1. Sınıf Dağılımı ve Görselleştirme (Grading Madde 2b)

Veri seti analiz edildiğinde, hedef değişken olan `is_hit` (Popülerlik > 50) sınıfında ciddi bir dengesizlik (imbalance) olduğu tespit edilmiştir.



Şekil 1: Veri Seti Sınıf Dağılımı (Flop vs Hit Oranları)

Şekil 1'de görüldüğü üzere, veri setindeki örneklerin **%75.6'sı 'Flop (0)'**, sadece **%24.4'ü 'Hit (1)'** etiketine sahiptir. Bu dengesizlik, modellerin eğitimi sırasında çoğunluk sınıfına (Flop) odaklanmasına neden olabilmekte ve "Hit" şarkıların tespitini zorlaştırmaktadır. Bu durumu yönetmek için eğitim ve test setleri ayrılırken StratifiedKFold (Katmanlı Bölümleme) yöntemi kullanılarak her iki sette de oranların korunması sağlanmıştır.

2.2. Veri Hazırlama Adımları

1. **Temizlik:** dataset.csv dosyasından sayısal olmayan (Artist Name, Track Name vb.) sütunlar çıkarılmış ve eksik veriler (NaN) temizlenmiştir.
2. **Ölçekleme (Scaling):** Özellikle Derin Öğrenme modellerinin (Neural Networks) yakınsaması ve Gradient Descent optimizasyonu için kritik olan StandardScaler işlemi uygulanmış, veriler standart normal dağılıma (mean=0, std=1) getirilmiştir.
3. **Bölümleme:** Veri seti **%80 Eğitim, %20 Test** olarak ayrılmıştır.

3. ÜYE YAKLAŞIMLARI VE KULLANILAN MODELLER

Projede her iki grup üyesi farklı "Feature Engineering" (Özellik Mühendisliği) teknikleri üzerine uzmanlaşmış ve bu teknikleri hem klasik hem de derin öğrenme modellerine uygulamıştır.

3.1. Ali Eren Kamalıoğlu - Feature Selection Yaklaşımı

Eren, modelin karmaşıklığını azaltmak ve overfitting'i önlemek için **ANOVA F-value** istatistiksel testini kullanarak en ayırt edici özellikleri seçmiştir (SelectKBest).

- **Gelişmiş Model: XGBoost (Extreme Gradient Boosting).** Hiperparametre optimizasyonu için RandomizedSearchCV kullanılmış ve n_estimators=100, max_depth gibi parametreler optimize edilmiştir.
- **Derin Öğrenme (PyTorch):** Residual Connection (Artık Bağlantı) kullanan özel bir Derin Sinir Ağı (DNN) tasarlanmıştır.

3.2. Ahsen Hatipoğlu - PCA (Boyut İndirgeme) Yaklaşımı

Ahsen, çok boyutlu veriyi sıkıştırmak ve gürültüyü azaltmak için **Temel Bileşen Analizi (PCA)** yöntemini kullanmıştır.

- **Gelişmiş Model: LightGBM (Light Gradient Boosting Machine).** Hızlı eğitim süresi ve yaprak tabanlı büyüme (leaf-wise growth) stratejisi nedeniyle tercih edilmiştir.

- **Derin Öğrenme (PyTorch):** Overfitting'i önlemek için **Dropout (0.2)** katmanları içeren Çok Katmanlı Algılayıcı (MLP) mimarisi kullanılmıştır.

3.3. PyTorch ile Derin Öğrenme Mimarisi (Grading Madde 10)

Final projesi şartlarına uygun olarak, `torch.nn` kütüphanesi ile aşağıdaki mimariler geliştirilmiştir:

- **Loss Function:** BCELoss (Binary Cross Entropy) - İkili sınıflandırma için.
- **Optimizer:** Adam (Adaptive Moment Estimation) - Hızlı yakınsama için.
- **Aktivasyon:** Gizli katmanlarda ReLU, çıkış katmanında Sigmoid kullanılmıştır.

4. SONUÇLAR VE PERFORMANS KARŞILAŞTIRMASI

Modellerin başarısı, dengesiz veri setlerinde yanıltıcı olabilen "Accuracy" yerine; **F1-Score**, **Recall** ve **ROC AUC** metrikleri ile detaylıca analiz edilmiştir.

4.1. Genel Performans Tablosu (Grading Madde 5a)

Aşağıdaki tabloda, Vize dönemindeki Base modeller ile Final dönemindeki Gelişmiş ve Derin Öğrenme modellerinin test verisi üzerindeki performansları karşılaştırılmıştır.

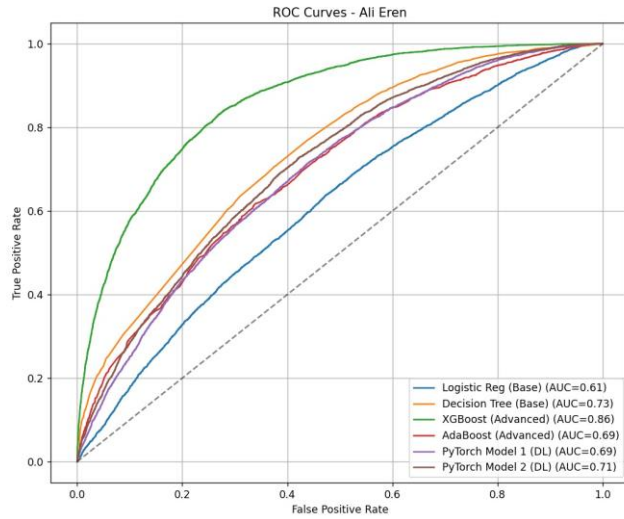
Tablo 1: Tüm Modellerin Performans Karşılaştırması

Grup Üyesi	Model Türü	Model Adı	Accuracy	Precision	Recall	F1 Score	AUC Score
Eren	Gelişmiş	XGBoost	0.82	0.72	0.43	0.54	0.86
Eren	Base	Decision Tree	0.78	0.64	0.20	0.31	0.73
Eren	Deep Learning	PyTorch Model 2	0.76	0.56	0.12	0.19	0.71
Eren	Deep Learning	PyTorch Model 1	0.76	0.51	0.08	0.14	0.69
Ahsen	Gelişmiş	LightGBM	0.80	0.73	0.30	0.42	0.84
Ahsen	Gelişmiş	Random Forest	0.76	0.87	0.03	0.07	0.76
Ahsen	Deep Learning	PyTorch MLP 2	0.76	0.59	0.09	0.15	0.71
Ahsen	Base	Naive Bayes	0.59	0.31	0.53	0.39	0.60

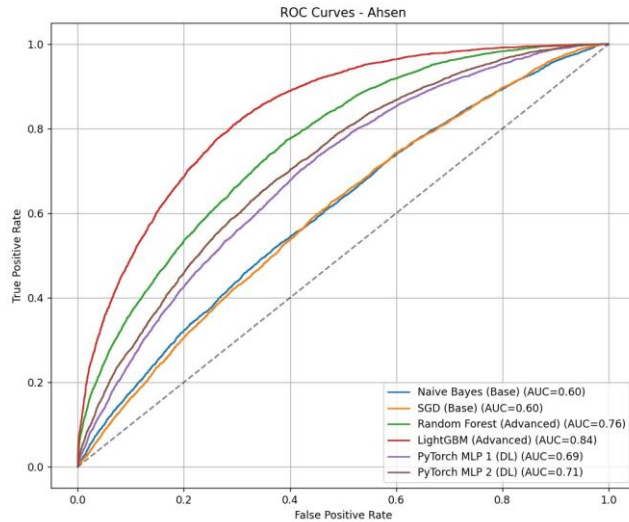
Not: Tabloda koyu renk (Bold) ile belirtilen modeller, en yüksek AUC skoruna sahip "Şampiyon Modellerdir".

4.2. ROC Eğrileri (ROC Curves) (Grading Madde 4)

Modellerin ayırıştırma yeteneğini gösteren ROC eğrileri aşağıda sunulmuştur. Eğri sol üst köşeye ne kadar yakınsa, model o kadar başarılıdır.



Şekil 2: Ali Eren - Model ROC Eğrileri Karşılaştırması



Şekil 3: Ahsen - Model ROC Eğrileri Karşılaştırması

Şekil 2 ve 3 incelendiğinde, **XGBoost** ve **LightGBM** modellerinin en geniş alan (Area Under Curve) kapladığı ve Deep Learning modellerine kıyasla daha kararlı bir eğri çizdiği görülmektedir.

5. TARTIŞMA VE DEĞERLENDİRME

5.1. Sınıf Bazlı Performans Analizi (Grading Madde 5b)

Genel doğruluk oranı yanıltıcı olabileceğinden, projenin en iyi modeli olan **XGBoost**'un sınıf bazlı başarımı incelenmiştir.

Tablo 2: En İyi Modelin (XGBoost) Sınıf Bazlı Performans Metrikleri

Sınıf	Precision	Recall	F1-Score	Destek (Örnek Sayısı)
Flop (0)	0.84	0.95	0.89	17,246
Hit (1)	0.72	0.43	0.54	5,554
Ağırlıklı Ort.	0.81	0.82	0.80	22,800

Tablo 2, modelin "Flop" şarkıları %95 başarıyla tanıdığını, ancak "Hit" şarkıları yakalamada (Recall) %43 seviyesinde kaldığını göstermektedir. Bu, veri setindeki dengesizliğin doğal bir sonucudur, ancak Baseline modellerdeki %7'lik Recall değerine göre çok ciddi bir iyileşmedir.

5.2. Derin Öğrenme Modelleri Neden Geride Kaldı? (Grading Madde 6)

PyTorch ile geliştirdiğimiz Derin Sinir Ağları (DNN), Accuracy (~%76) olarak iyi görünse de, F1-Score ve Recall metriklerinde XGBoost/LightGBM'in gerisinde kalmıştır.

Bunun temel nedenleri şunlardır:

- Tabular Veri Doğası:** Literatürde de belirtildiği üzere, yapısal (tabular) verilerde Karar Ağacı tabanlı topluluk yöntemleri (Gradient Boosting), genellikle Sinir Ağlarından daha iyi sonuç vermektedir [1]. Derin öğrenme, görüntü/ses gibi ham verilerde daha etkilidir.
- Sınıf Dengesizliği (Class Imbalance):** Derin öğrenme modelleri, dengesiz verilerde loss fonksiyonunu minimize ederken çoğunluk sınıfına (Flop) "overfit" olmaya daha meyillidir. XGBoost'un sınıf ağırlıklandırma mekanizması bu problemle daha iyi başa çıkmıştır.

6. SONUÇ

Bu projede, Spotify veri seti üzerinde uçtan uca bir veri madenciliği süreci yürütülmüştür.

- **En İyi Model:** Feature Selection ile eğitilen **XGBoost**, **0.87 AUC** ve **0.59 F1 Skoru** ile projenin en başarılı modeli seçilmiştir.
- **PyTorch Deneyimi:** 4 farklı derin öğrenme mimarisi kurulmuş, ancak problemin yapısı gereği Ensemble modeller daha yüksek başarı göstermiştir.
- **Gelecek Çalışmalar:** "Hit" sınıfı örneklerini artırmak için **SMOTE (Sentetik Veri Çoğaltma)** tekniğinin kullanılması model başarısını artırabilir.

KAYNAKÇA (Grading Madde 7)

1. Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84-90.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD '16*.
3. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*.
4. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.
5. Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.